МАТЕРИАЛЫ КОНТРОЛЬНОЙ ТОЧКИ

к курсовой работе по дисциплине «Основы программирования»

«Разработка программы для интегрирования функций, заданных формулой»

Вариант 7. Студент: Кириенко Артём Максимович, группа БИ-25. Руководитель: Самойлов Д.В. Этапы календарного плана: 1 – описание задачи и содержательная блок-схема; 2 – переменные, константы и подпрограммы; 3 – интерфейс программы; 4 – программный код основного алгоритма. Язык программирования – C# (.NET Framework 4.8, Windows Forms). Исходный код: репозиторий progrm_kr, каталог src/Integral.Core.

1 Описание задачи

1.1 Постановка задачи

Необходимо разработать программу, которая вычисляет определённый интеграл функции одной переменной, заданной формулой в текстовом виде:

$$I = \int f(x)\,dx,\quad x \in [a; b] \qquad (1.1)$$

Исходные данные программа читает из текстового файла *.txt или принимает из полей ввода. Для вычисления пользователь выбирает один из двух методов численного интегрирования – метод трапеций или метод Симпсона – либо запускает оба метода для сравнения. Результаты выводятся в окно программы и сохраняются в файл *.html.

1.2 Исходные данные

Файл исходных данных состоит из строк вида «ключ = значение»; символ «#» начинает комментарий. Ключи:

– function – формула подынтегральной функции f(x) (обязательный);

– a, b – нижний и верхний пределы интегрирования (обязательные);

– method – метод: trapezoid, simpson или both (по умолчанию simpson);

– n – число разбиений отрезка [a; b] (по умолчанию 100);

– eps – требуемая точность; если задана, число разбиений подбирается автоматически.

В формуле допускаются числа (разделитель целой и дробной части – точка или запятая), переменная x, константы pi и e, знаки + – * / ^, круглые скобки и функции sin, cos, tg, ctg, arcsin, arccos, arctg, exp, ln, lg, sqrt, abs. Знак умножения нужно писать явно: «2*x», а не «2x».

1.3 Результаты работы программы

Для каждого выбранного метода программа выводит: значение интеграла; итоговое число разбиений n и шаг h; оценку погрешности по правилу Рунге; число вычислений подынтегральной функции; время расчёта; таблицу сходимости (при подборе n по заданной точности); график функции с закрашенной площадью. При некорректных данных выводится сообщение с указанием позиции ошибки в формуле, значения x или номера строки файла.

1.4 Расчётные формулы

Отрезок [a; b] делится на n равных частей с шагом $h = (b - a) / n$, узлы $x_i = a + i \cdot h$, значения $f_i = f(x_i)$. Составная формула трапеций (порядок точности 2):

$$I \approx h \cdot [(f_0 + f_n) / 2 + f_1 + f_2 + \ldots + f_{n-1}] \quad (1.2)$$

Составная формула Симпсона (порядок точности 4, n – чётное):

$$I \approx h / 3 \cdot [f_0 + 4 \cdot (f_1 + f_3 + \ldots + f_{n-1}) + 2 \cdot (f_2 + f_4 + \ldots + f_{n-2}) + f_n] \quad (1.3)$$

Погрешность оценивается по правилу Рунге: вычисляются приближения I_n и I_{2n}, и

$$R \approx | I_{2n} - I_n | / (2^p - 1), \; p = 2 \text{ (трапеции)}, \; p = 4 \text{ (Симпсон)} \quad (1.4)$$

Если задана точность ε, число разбиений удваивается, пока $R > \varepsilon$ и $n < 2^{20} = 1\,048\,576$.

1.5 Особенности и ограничения

Метод Симпсона точен для многочленов до третьей степени. Оценка по правилу Рунге верна только для достаточно гладких функций. Пример из задания $f(x) = 20 \cdot \sin(3\sqrt{x})$ на отрезке [0; 10] негладкий: производная функции при x = 0 бесконечна, поэтому порядок сходимости падает до 1,5 вместо 2 и 4. Проверка по аналитической первообразной $40 \cdot (\sin 3t / 9 - t \cdot \cos 3t / 3)$, $t = \sqrt{x}$, даёт точное значение 41,806922543. Метод Симпсона при $\varepsilon = 10^{-6}$ выдаёт 41,806917842 (фактическая ошибка $4{,}7 \cdot 10^{-6}$), а оценка по Рунге составляет всего $5{,}7 \cdot 10^{-7}$. Поэтому программа определяет фактический порядок сходимости по трём последним удвоениям и предупреждает пользователя, если он ниже теоретического более чем на 20 %.

Вычисление прекращается с сообщением об ошибке при делении на ноль, логарифме неположительного числа, корне из отрицательного числа, арксинусе вне отрезка [−1; 1], вычислении тангенса в точке разрыва, степени отрицательного числа с нецелым показателем и любом нечисловом результате.

2 Содержательная блок-схема

2.1 Общий алгоритм работы программы

Общая блок-схема программы приведена на рисунке 2.1. Пользователь вводит исходные данные из файла или с формы; данные проверяются, формула

разбирается в дерево выражения, выбирается метод и вычисляется интеграл. При ошибке на любом этапе выводится сообщение, и ввод повторяется. Результаты выводятся на форму и по желанию сохраняются в HTML-отчёт.

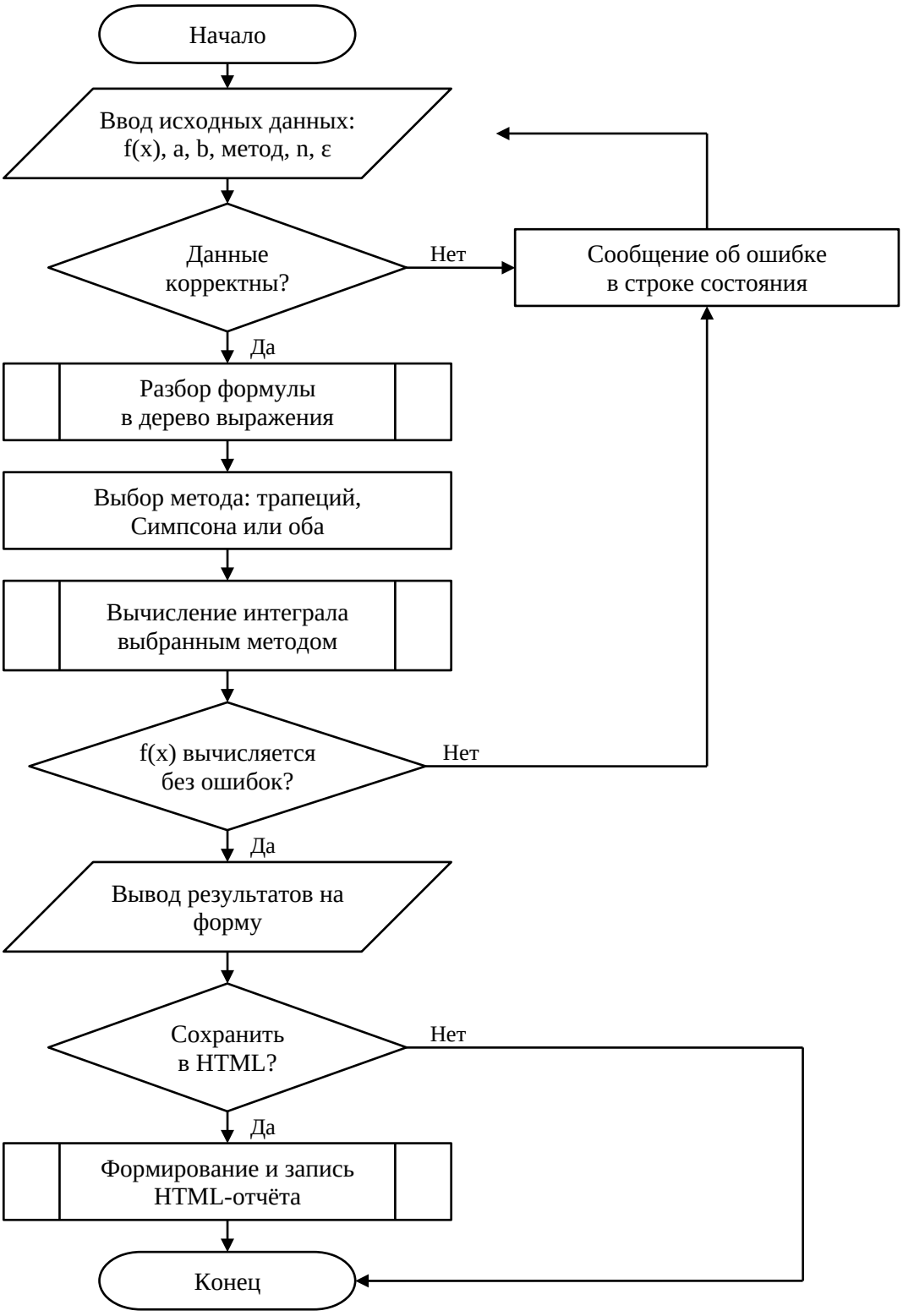


Рисунок 2.1 – Общая блок-схема алгоритма программы

2.2 Алгоритм вычисления интеграла

Блок-схема функции Integrator.Integrate приведена на рисунке 2.2. Через Q(n) обозначена формула выбранного метода – (1.2) или (1.3). При ε = 0 вычисляется

значение при заданном n и оценка погрешности по формуле (1.4). При ε > 0 число разбиений удваивается до достижения точности либо предела 2^{20}; каждое удвоение записывается в таблицу сходимости.

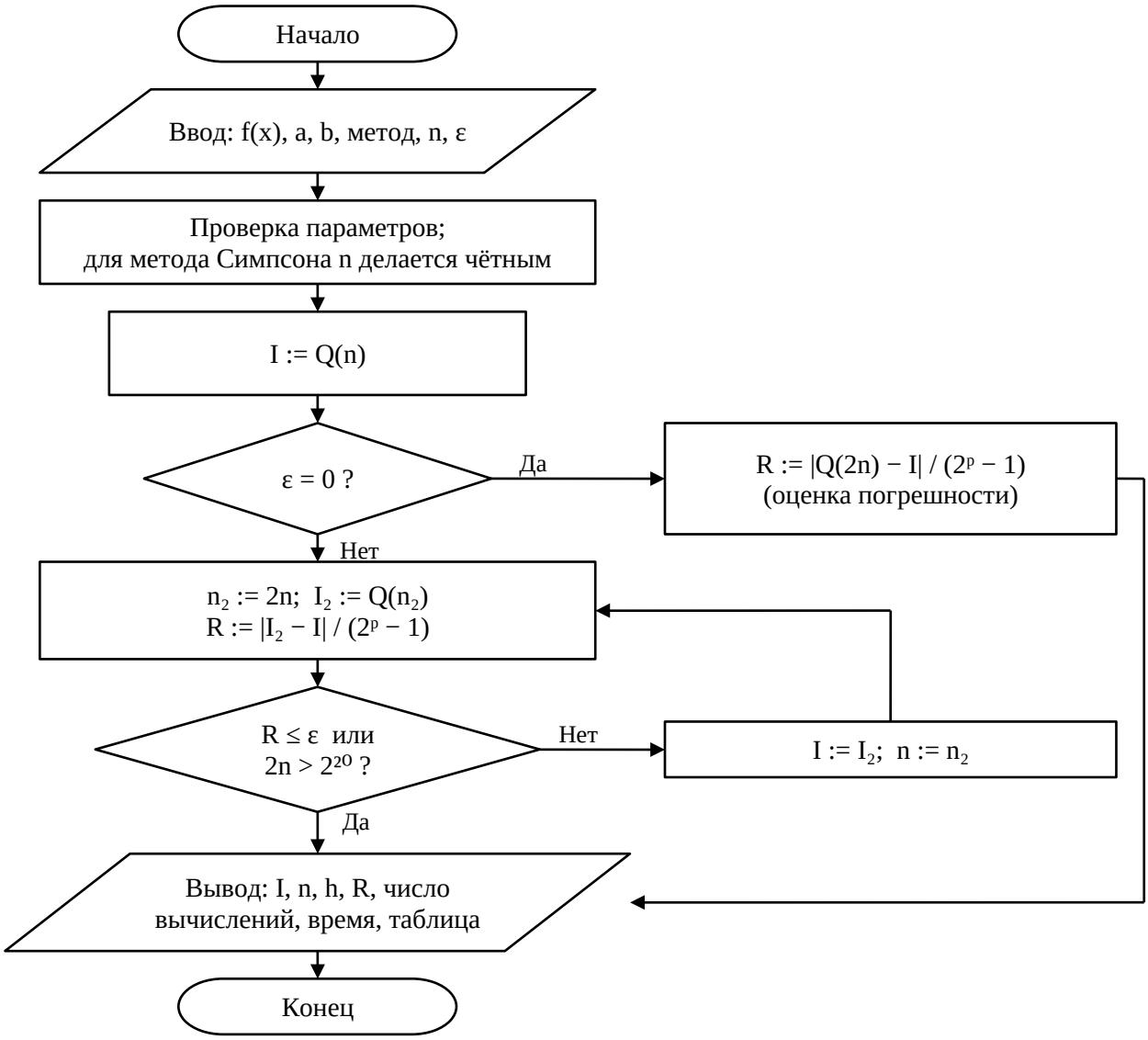


Рисунок 2.2 – Блок-схема алгоритма вычисления интеграла

3 Переменные, константы и подпрограммы

Таблицы составлены по исходному коду и сверены с ним автоматически: каждое имя из таблиц присутствует в программе. Для локальных переменных приведены только основные.

3.1 Переменные

Таблица 3.1 – Переменные программы

Имя	Диапазон значений	Тип данных	Описание
`Lexer._src`	строка	string	Анализируемая формула
`Lexer._pos`	0…длина формулы	int	Индекс следующего символа формулы
`Token.Type`	Number…End (12 значений)	TokenType	Тип лексемы

Имя	Диапазон значений	Тип данных	Описание
`Token.Position`	1…длина+1	int	Позиция лексемы в формуле, считая с 1
`Token.Length`	0…длина формулы	int	Длина лексемы в символах
`Token.Value`	любое double	double	Значение числа или константы (pi, e)
`Token.Name`	строка	string	Имя функции или константы
`Parser._lexer`	объект	Lexer	Источник лексем
`Parser._cur`	лексема	Token	Текущая лексема разбора
`Parser._depth`	0…200	int	Текущая вложенность разбора
`IntegrationParams.A`	конечное число	double	Нижний предел интегрирования
`IntegrationParams.B`	конечное число	double	Верхний предел интегрирования
`IntegrationParams.N`	1…1 048 576	int	Число разбиений (начальное, если задана точность)
`IntegrationParams.Eps`	≥ 0, конечное	double	Требуемая точность; 0 — расчёт с фиксированным n
`IntegrationResult.Value`	любое double	double	Значение интеграла
`IntegrationResult.N`	1…1 048 576	int	Итоговое число разбиений
`IntegrationResult.H`	любое double	double	Итоговый шаг (B − A) / N
`IntegrationResult.ErrorEstimate`	≥ 0 или NaN	double	Оценка погрешности по правилу Рунге
`IntegrationResult.ObservedOrder`	double или NaN	double	Фактический порядок сходимости
`IntegrationResult.OrderDegraded`	False…True	bool	Фактический порядок ниже теоретического (оценка занижена)
`IntegrationResult.Evaluations`	$0…2^{63}-1$	long	Число вычислений подынтегральной функции
`IntegrationResult.ElapsedMs`	≥ 0	double	Время расчёта, мс
`IntegrationResult.Converged`	False…True	bool	Достигнута ли заданная точность
`IntegrationResult.NAdjusted`	False…True	bool	n увеличено до чётного (метод Симпсона)
`IntegrationResult.Steps`	список	IReadOnlyList<ConvergenceStep>	Таблица сходимости
`ConvergenceStep.ErrorEstimate`	≥ 0 или NaN	double	Оценка погрешности на шаге (NaN на первом)
`IntegrationTask.Formula`	строка	string	Формула подынтегральной функции
`IntegrationTask.Method`	Trapezoid, Simpson, Both	MethodChoice	Выбранный метод
`EvalException.X`	любое double	double	Значение x, при котором возникла ошибка
`FormulaException.Position`	1…длина+1	int	Позиция ошибки в формуле
`TaskFileException.Line`	0…число строк	int	Номер строки файла с ошибкой (0 — файл в целом)
`Integrator.evals`	$0…2^{63}-1$	long	Счётчик вычислений f(x) в Integrate

Имя	Диапазон значений	Тип данных	Описание
`Integrator.order`	2 или 4	int	Порядок точности выбранного метода
`Integrator.divisor`	3 или 15	double	Знаменатель $2^p - 1$ в правиле Рунге
`Integrator.prev`	любое double	double	Значение интеграла на предыдущем шаге
`Integrator.cur`	любое double	double	Значение интеграла при удвоенном n
`Integrator.n2`	2…1 048 576	int	Удвоенное число разбиений
`Integrator.steps`	список	List<ConvergenceStep>	Накопленная таблица сходимости
`Integrator.h`	любое double	double	Шаг разбиения в формулах трапеций и Симпсона
`Integrator.sum`	любое double	double	Накопленная сумма значений функции
`TaskFile.seen`	словарь	Dictionary<string,int>	Ключи, уже встреченные в файле, и номера строк
`TaskFile.lineNo`	1…число строк	int	Номер разбираемой строки файла

3.2 Константы и перечисления

Таблица 3.2 – Константы и перечисления программы

Имя	Значение	Тип данных	Описание
`IntegrationParams.MaxN`	1 << 20 = 1 048 576	int	Предел удвоения числа разбиений
`Parser.MaxDepth`	200	int	Предельная вложенность разбора (защита от переполнения стека)
`Functions.PoleEps`	10^{-12}	double	Порог близости к полюсу для tg и ctg
`Functions.Table`	12 функций	Dictionary<string, Func<double,double>>	sin, cos, tg, ctg, arcsin, arccos, arctg, exp, ln, lg, sqrt, abs
`Functions.Constants`	pi, e	Dictionary<string,double>	Именованные константы формулы
`TaskFile.Keys`	6 ключей	string[]	function, a, b, method, n, eps
`Ui.ControlHeight`	20	int	Высота поля ввода, пикс. (методические указания, табл. 2.1)
`Ui.GapRelated`	5	int	Интервал между связанными элементами, пикс.
`Ui.GapUnrelated`	8	int	Интервал между несвязанными элементами, пикс.
`PlotPanel.MaxNodesShown`	64	int	Наибольшее n, при котором на графике показываются узлы разбиения
`TokenType`	12 значений	перечисление	Number, Variable, Constant, Function, Plus, Minus, Multiply, Divide, Power, LeftParen, RightParen, End

Имя	Значение	Тип данных	Описание
`Method`	2 значения	перечисление	Trapezoid (порядок 2), Simpson (порядок 4)
`MethodChoice`	3 значения	перечисление	Trapezoid, Simpson, Both
`ErrorCode`	27 значений	перечисление	Коды ошибок разбора, вычисления, параметров и файла (тексты — в Messages)

3.3 Подпрограммы

Таблица 3.3 – Подпрограммы ядра

Подпрограмма	Параметры	Результат	Назначение
`Lexer.Next`	—	Token	Выделяет следующую лексему формулы
`Lexer.ReadNumber`	—	Token	Читает число (целое, дробное, с порядком)
`Lexer.ReadWord`	—	Token	Читает слово: x, константа или имя функции
`Parser.Parse`	string formula	Node	Разбирает формулу в дерево выражения
`Parser.UsesX`	Node node	bool	Проверяет, входит ли x в выражение
`Parser.ParseExpression`	—	Node	Сумма и разность слагаемых
`Parser.ParseTerm`	—	Node	Произведение и частное множителей
`Parser.ParseFactor`	—	Node	Множитель со знаком
`Parser.ParsePower`	—	Node	Возведение в степень (правая ассоциативность)
`Parser.ParseBase`	—	Node	Число, x, константа, функция или скобки
`Parser.Expect`	TokenType type	—	Проверяет ожидаемую лексему и переходит к следующей
`Evaluator.Evaluate`	Node node, double x	double	Вычисляет дерево выражения в точке x
`Evaluator.Binary`	BinaryNode b, double x	double	Вычисляет бинарную операцию с проверками
`Evaluator.Function`	FunctionNode f, double x	double	Вычисляет функцию с проверкой области определения
`Evaluator.Finite`	double v, double x	double	Проверяет, что результат — конечное число
`Functions.IsFunction`	string name	bool	Проверяет, что имя — встроенная функция
`Functions.TryGetConstant`	string name, out double value	bool	Возвращает значение константы по имени
`Functions.TryApply`	string name, double arg, out double result	ErrorCode	Вычисляет функцию, проверяя область определения
`Integrator.Integrate`	Node f, IntegrationParams p, Method method	IntegrationResult	Основной алгоритм: интегрирование с подбором n по точности
`Integrator.Trapezoid`	Func f, double a, double b, int n	double	Составная формула трапеций
`Integrator.Simpson`	Func f, double a, double b, int n	double	Составная формула Симпсона (n чётное)

Подпрограмма	Параметры	Результат	Назначение
`Integrator.Validate`	IntegrationParams p	—	Проверка пределов, n и точности
`Integrator.ObserveOrder`	IntegrationResult res, IList steps, int order	—	Определяет фактический порядок сходимости
`TaskFile.Parse`	string text	IntegrationTask	Разбирает файл исходных данных
`TaskFile.ToText`	IntegrationTask t	string	Формирует текст файла исходных данных
`TaskFile.Assign`	IntegrationTask task, string key, string value, int line	—	Присваивает значение ключа с проверкой
`TaskFile.ReadNumber`	string value, string key, int line	double	Читает число из значения ключа
`TaskFile.ReadMethod`	string value, string key, int line	MethodChoice	Читает название метода
`NumberParser.TryParse`	string text, out double value	bool	Читает число из поля ввода (точка или запятая)
`AxisScale.NiceStep`	double min, double max, int target	double	Подбирает шаг делений оси из ряда 1, 2, 5 x 10^k
`AxisScale.Ticks`	double min, double max, int target	IList<double>	Значения делений оси внутри диапазона
`CalcForm.TryReadTask`	out IntegrationTask task, out Node node	bool	Читает и проверяет поля окна «Расчёт»
`CalcForm.Calculate`	—	—	Выполняет расчёт выбранным методом (или обоими) и выводит результаты
`CalcForm.LoadTask`	IntegrationTask t	—	Заполняет поля ввода из файла исходных данных
`CalcForm.ShowResults`	—	—	Выводит сводную таблицу и предупреждения
`CalcForm.ShowSteps`	—	—	Выводит таблицу сходимости выбранного метода
`PlotPanel.OnPaint`	PaintEventArgs e	—	Рисует график: сетку, площадь, узлы, кривую
`PlotPanel.DrawGrid`	Graphics g, Rectangle plot, ...	—	Рисует сетку и подписи осей
`MainForm.NewCalc`	IntegrationTask task, string title	CalcForm	Открывает новое окно расчёта
`MainForm.OpenData`	—	—	Загружает файл исходных данных в новое окно
`MainForm.SaveData`	—	—	Сохраняет исходные данные в файл *.txt
`MainForm.ShowGraph`	—	—	Открывает окно с графиком
`MainForm.ShowHelp`	int section	—	Открывает справку на нужном разделе
`MainForm.UpdateCommands`	—	—	Включает и выключает команды меню и панели инструментов
`IconFactory.Create`	Kind kind, int size	Bitmap	Рисует значок панели инструментов
`Messages.Text`	ErrorCode code, params object[] args	string	Возвращает готовую русскую фразу об ошибке
`Messages.Num`	double value	string	Форматирует число для сообщений

4 Интерфейс программы (макет)

Интерфейс многооконный (MDI). Главное окно содержит главное меню, панель инструментов и строку состояния; дочерние окна открываются внутри главного. Размеры и цвета элементов выбираются по методическим указаниям: фон формы светло-серый, поля ввода белые, шрифт Tahoma 10 пт, высота элементов 20 пикселей, интервалы 5 и 8 пикселей, выравнивание по левому краю, группы элементов в контейнерах с заголовками, подсказки к элементам. Таблица результатов и график размещаются в разных окнах.

4.1 Окна программы

Таблица 4.1 – Окна программы

Окно	Тип	Назначение
Интегрирование функций	главное, MDI	Меню, панель инструментов, строка состояния; контейнер дочерних окон
Расчёт	дочернее, несколько	Ввод формулы, пределов, метода и точности; результаты и таблица сходимости
График	дочернее	Кривая f(x), площадь под ней, узлы разбиения, оси с подписями, автомасштаб
Справка	немодальное	Работа с программой: разделы, формат файла, синтаксис формул, сообщения об ошибках
О программе	модальное	Вуз, кафедра, группа, автор, тема, руководитель, год; описание программы

4.2 Главное меню

Таблица 4.2 – Структура главного меню

Меню	Пункт	Клавиши	Кнопка панели
Файл	Новый расчёт	Ctrl+N	есть
	Открыть данные…	Ctrl+O	есть
	Сохранить данные…	Ctrl+S	есть
	Экспорт отчёта в HTML… (обработчик — этап 5)	Ctrl+E	есть
	Выход	Alt+F4	—
Расчёт	Вычислить	F5	есть
	Метод: трапеций / Симпсона / оба	—	—
	Показать график	F6	есть
Окно	Каскадом, Мозаикой, Упорядочить значки, Закрыть все; список открытых окон	—	—
Справка	Содержание	F1	есть
	Формат входного файла	—	—
	О программе	—	есть

4.3 Панель инструментов и строка состояния

Панель инструментов дублирует основные команды меню: «Новый расчёт», «Открыть данные», «Сохранить данные», «Экспорт в HTML», «Вычислить», «График», «Справка», «О программе». Недоступные в текущем состоянии кнопки (например, «Вычислить» при отсутствии открытых окон расчёта) блокируются.

Строка состояния состоит из трёх областей: подсказка к элементу под указателем мыши, состояние последнего расчёта, выбранный метод.

4.4 Окно «Расчёт»

Таблица 4.3 – Элементы окна «Расчёт»

Элемент	Тип	Назначение
`grpInput`	GroupBox	Контейнер «Функция и пределы»
`txtFormula`	TextBox	Формула f(x)
`txtLowerLimit`, `txtUpperLimit`	TextBox	Пределы интегрирования a и b
`grpMethod`	GroupBox	Контейнер «Метод и точность»
`cmbMethod`	ComboBox	Метод: трапеций, Симпсона, оба метода
`txtN, txtEps`	TextBox	Число разбиений n; точность ε (пусто – не задана)
`btnCalculate`	Button	Запуск расчёта (F5)
`grpResult`	GroupBox	Контейнер «Результат»
`dgvSummary`	DataGridView	Сводная таблица: метод, интеграл, n, h, оценка погрешности, порядок, число вычислений f(x), время
`lblWarning`	Label	Предупреждения: нечётное n, недостигнутая точность, негладкая функция
`dgvConvergence`	DataGridView	Таблица сходимости выбранного метода: n, значение, оценка погрешности

Формы реализованы в проекте Integral.App (этап 3 календарного плана): MainForm, CalcForm, GraphForm, HelpForm, AboutForm. Команда экспорта отчёта в HTML присутствует в меню и на панели инструментов; её обработчик реализуется на этапе 5. Ошибки ввода сопровождаются сообщением и переходом курсора в ошибочное поле; при ошибке в формуле курсор ставится на позицию ошибки. Подсказки элементов выводятся в строке состояния. Снимки окон будут приведены в приложении «Формы программы» пояснительной записки.

Приложение А
Программный код основного алгоритма

Модуль Lexer.cs

```
using System;
using System.Globalization;

namespace Integral.Core
{
    /// <summary>Лексический анализатор формулы: выделяет лексемы по одной.</summary>
    public sealed class Lexer
    {
        private readonly string _src;
        private int _pos; // индекс следующего символа, с 0

        public Lexer(string source)
        {
            _src = source ?? string.Empty;
        }

        /// <summary>Возвращает следующую лексему; в конце строки — <see
cref="TokenType.End"/>.</summary>
        public Token Next()
        {
            while (_pos < _src.Length && char.IsWhiteSpace(_src[_pos])) _pos++;
            if (_pos >= _src.Length) return new Token(TokenType.End, _pos + 1, 0);

            char c = _src[_pos];
            int pos1 = _pos + 1;
            switch (c)
            {
                case '+': _pos++; return new Token(TokenType.Plus, pos1, 1);
                case '-': _pos++; return new Token(TokenType.Minus, pos1, 1);
                case '*': _pos++; return new Token(TokenType.Multiply, pos1, 1);
                case '/': _pos++; return new Token(TokenType.Divide, pos1, 1);
                case '^': _pos++; return new Token(TokenType.Power, pos1, 1);
                case '(': _pos++; return new Token(TokenType.LeftParen, pos1, 1);
                case ')': _pos++; return new Token(TokenType.RightParen, pos1, 1);
            }
            if (IsDigit(c)) return ReadNumber();
            if (c == '.' || c == ',') throw new FormulaException(ErrorCode.BadNumber, pos1, pos1);
            if (IsLetter(c)) return ReadWord();
            throw new FormulaException(ErrorCode.BadChar, pos1, c, pos1);
        }

        /// <summary>Число: цифры [.|, цифры] [e|E [+|-] цифры].</summary>
        private Token ReadNumber()
        {
            int start = _pos;
            while (_pos < _src.Length && IsDigit(_src[_pos])) _pos++;
            if (_pos < _src.Length && (_src[_pos] == '.' || _src[_pos] == ','))
            {
                _pos++;
                if (_pos >= _src.Length || !IsDigit(_src[_pos]))
                    throw new FormulaException(ErrorCode.BadNumber, start + 1, start + 1);
                while (_pos < _src.Length && IsDigit(_src[_pos])) _pos++;
            }
            // Экспонента принимается, только если за ней идут цифры; иначе "e" — отдельная константа.
            if (_pos < _src.Length && (_src[_pos] == 'e' || _src[_pos] == 'E'))
            {
                int p = _pos + 1;
                if (p < _src.Length && (_src[p] == '+' || _src[p] == '-')) p++;
                if (p < _src.Length && IsDigit(_src[p]))
                {
                    while (p < _src.Length && IsDigit(_src[p])) p++;
                    _pos = p;
                }
            }
            string text = _src.Substring(start, _pos - start).Replace(',', '.');
            double value = double.Parse(text, NumberStyles.Float, CultureInfo.InvariantCulture);
            if (double.IsInfinity(value)) throw new FormulaException(ErrorCode.BadNumber, start + 1,
start + 1);
```

```
            return new Token(TokenType.Number, start + 1, _pos - start, value);
        }

        /// <summary>Слово: x, константа, имя функции; регистр не учитывается.</summary>
        private Token ReadWord()
        {
            int start = _pos;
            while (_pos < _src.Length && (IsLetter(_src[_pos]) || IsDigit(_src[_pos]))) _pos++;
            string raw = _src.Substring(start, _pos - start);
            string word = raw.ToLowerInvariant();
            int len = _pos - start;
            if (word == "x") return new Token(TokenType.Variable, start + 1, len);
            if (Functions.TryGetConstant(word, out double c)) return new Token(TokenType.Constant,
start + 1, len, c, word);
            if (Functions.IsFunction(word)) return new Token(TokenType.Function, start + 1, len, 0,
word);
            throw new FormulaException(ErrorCode.UnknownIdent, start + 1, raw, start + 1);
        }

        private static bool IsDigit(char c) => c >= '0' && c <= '9';
        private static bool IsLetter(char c) => (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
    }
}
```

Модуль Parser.cs

```
namespace Integral.Core
{
    /// <summary>
    /// Синтаксический анализатор формулы (рекурсивный спуск по грамматике БНФ).
    /// Выражение  ::= Слагаемое { ('+'|'-') Слагаемое }
    /// Слагаемое  ::= Множитель { ('*'|'/') Множитель }
    /// Множитель  ::= ['+'|'-'] Степень
    /// Степень    ::= Основа [ '^' Множитель ]
    /// Основа     ::= Число | 'x' | Константа | Функция '(' Выражение ')' | '(' Выражение ')'
    /// </summary>
    public sealed class Parser
    {
        /// <summary>Максимальная вложенность разбора (защита от переполнения стека).</summary>
        public const int MaxDepth = 200;

        private readonly Lexer _lexer;
        private Token _cur;
        private int _depth;

        private Parser(string source)
        {
            _lexer = new Lexer(source);
            _cur = _lexer.Next();
        }

        /// <summary>Разбирает формулу в дерево выражения. Бросает <see
cref="FormulaException"/>.</summary>
        public static Node Parse(string formula)
        {
            var parser = new Parser(formula);
            if (parser._cur.Type == TokenType.End)
                throw new FormulaException(ErrorCode.EmptyExpression, 1);
            Node root = parser.ParseExpression();
            if (parser._cur.Type != TokenType.End)
                throw new FormulaException(ErrorCode.UnexpectedToken, parser._cur.Position,
parser._cur.Position);
            return root;
        }

        /// <summary>Есть ли в дереве переменная x.</summary>
        public static bool UsesX(Node node)
        {
            switch (node)
            {
                case VariableNode _: return true;
                case NegateNode n: return UsesX(n.Operand);
                case BinaryNode b: return UsesX(b.Left) || UsesX(b.Right);
                case FunctionNode f: return UsesX(f.Argument);
```

```
            default: return false;
        }
    }

    private void Advance() { _cur = _lexer.Next(); }

    private void Enter()
    {
        if (++_depth > MaxDepth)
            throw new FormulaException(ErrorCode.TooDeep, _cur.Position, MaxDepth);
    }

    private void Leave() { _depth--; }

    /// <summary>Выражение: сложение и вычитание слагаемых.</summary>
    private Node ParseExpression()
    {
        Enter();
        Node left = ParseTerm();
        while (_cur.Type == TokenType.Plus || _cur.Type == TokenType.Minus)
        {
            char op = _cur.Type == TokenType.Plus ? '+' : '-';
            Advance();
            left = new BinaryNode(op, left, ParseTerm());
        }
        Leave();
        return left;
    }

    /// <summary>Слагаемое: умножение и деление множителей.</summary>
    private Node ParseTerm()
    {
        Node left = ParseFactor();
        while (_cur.Type == TokenType.Multiply || _cur.Type == TokenType.Divide)
        {
            char op = _cur.Type == TokenType.Multiply ? '*' : '/';
            Advance();
            left = new BinaryNode(op, left, ParseFactor());
        }
        return left;
    }

    /// <summary>Множитель: необязательный унарный знак перед степенью.</summary>
    private Node ParseFactor()
    {
        Enter();
        Node result;
        if (_cur.Type == TokenType.Minus) { Advance(); result = new NegateNode(ParseFactor()); }
        else if (_cur.Type == TokenType.Plus) { Advance(); result = ParseFactor(); }
        else result = ParsePower();
        Leave();
        return result;
    }

    /// <summary>Степень: основа и необязательный показатель (правая ассоциативность).</summary>
    private Node ParsePower()
    {
        Node b = ParseBase();
        if (_cur.Type == TokenType.Power)
        {
            Advance();
            return new BinaryNode('^', b, ParseFactor());
        }
        return b;
    }

    /// <summary>Основа: число, x, константа, функция или выражение в скобках.</summary>
    private Node ParseBase()
    {
        Token t = _cur;
        switch (t.Type)
        {
            case TokenType.Number:
            case TokenType.Constant:
```

```
                    Advance();
                    return new NumberNode(t.Value);
                case TokenType.Variable:
                    Advance();
                    return new VariableNode();
                case TokenType.Function:
                    Advance();
                    if (_cur.Type != TokenType.LeftParen)
                        throw new FormulaException(ErrorCode.ExpectedLParen, _cur.Position, t.Name,
_cur.Position);
                    Advance();
                    Node arg = ParseExpression();
                    Expect(TokenType.RightParen);
                    return new FunctionNode(t.Name, arg);
                case TokenType.LeftParen:
                    Advance();
                    Node inner = ParseExpression();
                    Expect(TokenType.RightParen);
                    return inner;
                default:
                    throw new FormulaException(ErrorCode.ExpectedOperand, t.Position, t.Position);
            }
        }

        private void Expect(TokenType type)
        {
            if (_cur.Type != type)
                throw new FormulaException(ErrorCode.ExpectedRParen, _cur.Position, _cur.Position);
            Advance();
        }
    }
}
```

Модуль Evaluator.cs

```
using System;

namespace Integral.Core
{
    /// <summary>Вычисление дерева выражения в точке x с контролем области определения.</summary>
    public static class Evaluator
    {
        /// <summary>Значение выражения при заданном x. Бросает <see cref="EvalException"/>.</summary>
        public static double Evaluate(Node node, double x)
        {
            switch (node)
            {
                case NumberNode n: return n.Value;
                case VariableNode _: return x;
                case NegateNode g: return -Evaluate(g.Operand, x);
                case BinaryNode b: return Binary(b, x);
                case FunctionNode f: return Function(f, x);
                default: throw new ArgumentException("Неизвестный тип узла.", nameof(node));
            }
        }

        private static double Binary(BinaryNode b, double x)
        {
            double l = Evaluate(b.Left, x);
            double r = Evaluate(b.Right, x);
            double res;
            switch (b.Op)
            {
                case '+': res = l + r; break;
                case '-': res = l - r; break;
                case '*': res = l * r; break;
                case '/':
                    if (r == 0) throw new EvalException(ErrorCode.DivByZero, x);
                    res = l / r;
                    break;
                case '^':
                    if (l == 0 && r < 0) throw new EvalException(ErrorCode.DivByZero, x);
                    if (l < 0 && r != Math.Floor(r)) throw new EvalException(ErrorCode.PowDomain, x);
                    res = Math.Pow(l, r);
```

```
                    break;
                default: throw new ArgumentException("Неизвестная операция.", nameof(b));
            }
            return Finite(res, x);
        }

        private static double Function(FunctionNode f, double x)
        {
            double arg = Evaluate(f.Argument, x);
            ErrorCode code = Functions.TryApply(f.Name, arg, out double res);
            if (code != ErrorCode.None) throw new EvalException(code, x);
            return Finite(res, x);
        }

        private static double Finite(double v, double x)
        {
            if (double.IsNaN(v) || double.IsInfinity(v)) throw new EvalException(ErrorCode.NotFinite,
x);
            return v;
        }
    }
}
```

Модуль Integrator.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics;

namespace Integral.Core
{
    /// <summary>Численное интегрирование: формулы трапеций и Симпсона, оценка точности по
Рунге.</summary>
    public static class Integrator
    {
        /// <summary>Вычисляет интеграл функции f на [A; B]. Бросает <see
cref="IntegralException"/>.</summary>
        public static IntegrationResult Integrate(Node f, IntegrationParams p, Method method)
        {
            Validate(p);
            var sw = Stopwatch.StartNew();
            long evals = 0;
            Func<double, double> fn = x => { evals++; return Evaluator.Evaluate(f, x); };

            int order = method == Method.Simpson ? 4 : 2;
            double divisor = Math.Pow(2, order) - 1;
            var res = new IntegrationResult { Method = method };

            int n = Math.Max(p.N, method == Method.Simpson ? 2 : 1);
            if (method == Method.Simpson && n % 2 == 1) { n++; res.NAdjusted = true; }
            if (p.Eps > 0 && n < 2) n = 2;

            if (p.A == p.B)
            {
                res.Value = 0; res.N = n; res.H = 0; res.ErrorEstimate = 0;
                res.ElapsedMs = sw.Elapsed.TotalMilliseconds;
                return res;
            }

            Func<int, double> compute = k => method == Method.Simpson ? Simpson(fn, p.A, p.B, k) :
Trapezoid(fn, p.A, p.B, k);
            var steps = new List<ConvergenceStep>();
            double prev = compute(n);
            steps.Add(new ConvergenceStep(n, prev, double.NaN));
            res.Value = prev; res.N = n;

            if (p.Eps <= 0)
            {
                // Фиксированное n: оценка погрешности по Рунге, если удвоение не выходит за предел.
                if (n * 2 <= IntegrationParams.MaxN)
                    res.ErrorEstimate = Math.Abs(compute(n * 2) - prev) / divisor;
            }
            else
            {
```

```
                res.Converged = false;
                while (n * 2 <= IntegrationParams.MaxN)
                {
                    int n2 = n * 2;
                    double cur = compute(n2);
                    double r = Math.Abs(cur - prev) / divisor;
                    steps.Add(new ConvergenceStep(n2, cur, r));
                    res.Value = cur; res.N = n2; res.ErrorEstimate = r;
                    if (r <= p.Eps) { res.Converged = true; break; }
                    prev = cur; n = n2;
                }
            }

            res.H = (p.B - p.A) / res.N;
            res.Steps = steps;
            ObserveOrder(res, steps, order);
            res.Evaluations = evals;
            res.ElapsedMs = sw.Elapsed.TotalMilliseconds;
            return res;
        }

        /// <summary>Фактический порядок: log2((I(n)−I(n/2)) / (I(2n)−I(n))) по трём последним
шагам.</summary>
        private static void ObserveOrder(IntegrationResult res, IList<ConvergenceStep> s, int order)
        {
            int k = s.Count;
            if (k < 3) return;
            double d1 = s[k - 2].Value - s[k - 3].Value;
            double d2 = s[k - 1].Value - s[k - 2].Value;
            if (d2 == 0 || d1 / d2 <= 0) return;
            res.ObservedOrder = Math.Log(d1 / d2, 2);
            res.OrderDegraded = res.ObservedOrder < 0.8 * order;
        }

        /// <summary>Составная формула трапеций: h·[(f0+fn)/2 + Σ fi].</summary>
        public static double Trapezoid(Func<double, double> f, double a, double b, int n)
        {
            double h = (b - a) / n;
            double sum = (f(a) + f(b)) / 2;
            for (int i = 1; i < n; i++) sum += f(a + i * h);
            return h * sum;
        }

        /// <summary>Составная формула Симпсона (n чётное): h/3·[f0 + 4Σf_нечёт + 2Σf_чёт +
fn].</summary>
        public static double Simpson(Func<double, double> f, double a, double b, int n)
        {
            double h = (b - a) / n;
            double sum = f(a) + f(b);
            for (int i = 1; i < n; i++) sum += (i % 2 == 1 ? 4 : 2) * f(a + i * h);
            return h / 3 * sum;
        }

        private static void Validate(IntegrationParams p)
        {
            if (double.IsNaN(p.A) || double.IsInfinity(p.A) || double.IsNaN(p.B) ||
double.IsInfinity(p.B))
                throw new ParamsException(ErrorCode.BadLimits);
            if (p.N < 1 || p.N > IntegrationParams.MaxN)
                throw new ParamsException(ErrorCode.BadN, IntegrationParams.MaxN);
            if (double.IsNaN(p.Eps) || double.IsInfinity(p.Eps) || p.Eps < 0)
                throw new ParamsException(ErrorCode.BadEps);
        }
    }
}
```